Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

Vikhyathi M **1BM24EC253**

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 – 2027

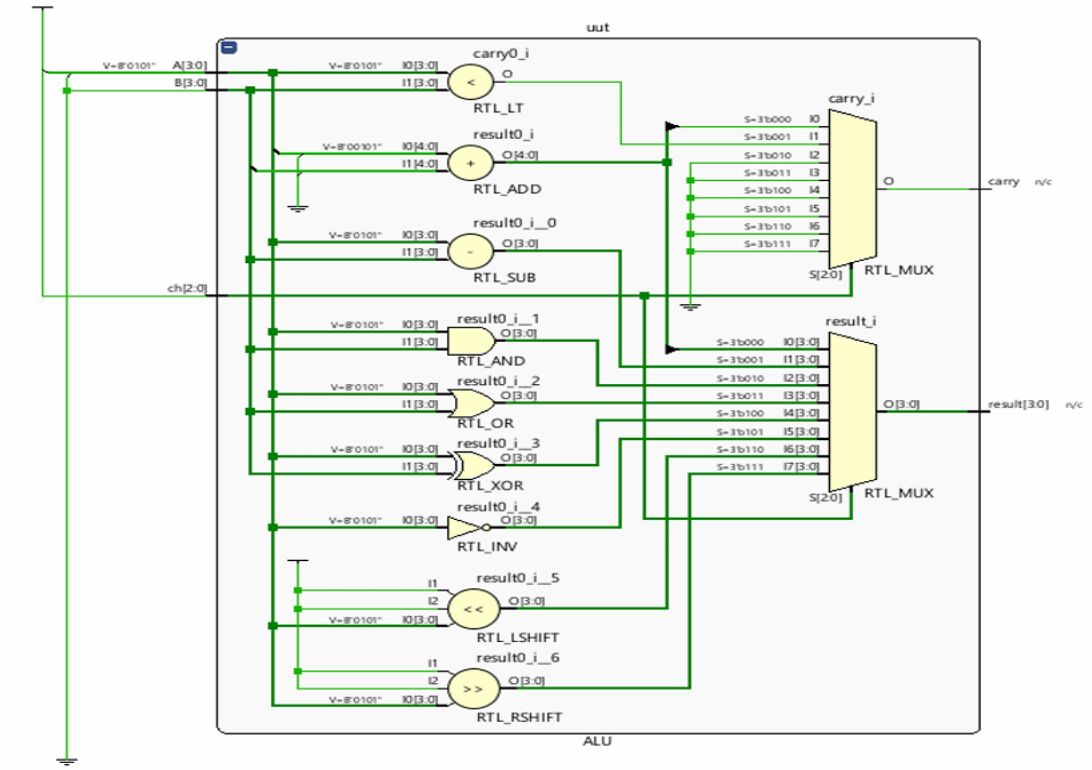
INDEX

ADLD AAT Report — Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL

S. No.	Contents	Page No.
1	Arithmetic Logic Unit (ALU)	3–4
2	Comparator	5–6
3	4-Bit Parity Generator and Checker	7–8
4	4-Bit Barrel Shifter	9–10
5	Timer Module	11–12
6	Ring Counter	13–14
7	Johnson Counter	15–16
8	Synchronous and Asynchronous Counters	17–20
9	Shift Register	21–22
10	Divide-by-N Counter	23–24
11	Mealy and Moore FSMs (one example each)	25–28
12	Random Sequence Generator	29–30

1. Arithmetic Logic Unit (ALU)

Design:



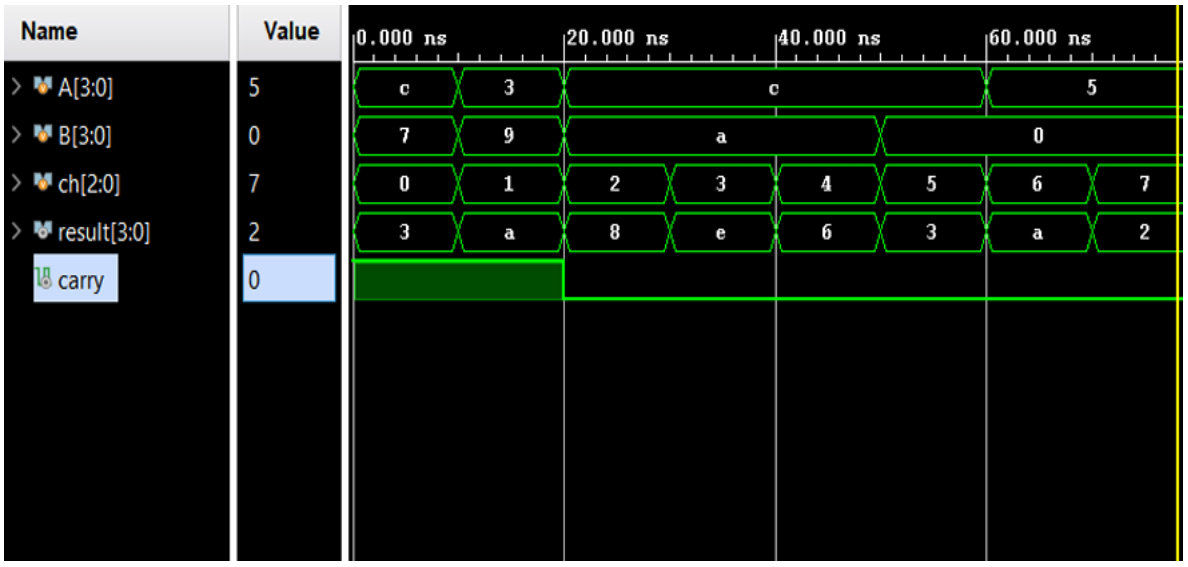
Code:

```
module ALU (input [3:0] A, B, input [2:0] ch, output reg [3:0] result, output reg carry);
always @(*) begin
result = 4'b0000;
carry = 1'b0;
case (ch)
//addition
3'b000: begin
{carry, result} = {1'b0, A} + {1'b0, B};
end
//subtraction
3'b001: begin
result = A - B;
carry = (A < B);
end
//AND operation
3'b010: begin
result = A & B;
end
//OR operation
3'b011: begin
result = A | B;
end
//XOR operation
3'b100: begin
result = A ^ B;
end
//NOT operation
3'b101: begin
result = ~A;
end
//LEFT SHIFT
3'b110: begin
result = A << 1;
end
//RIGHT SHIFT
3'b111: begin
result = A >> 1;
end
default: begin
result = 4'b0000;
carry = 1'b0;
end
endcase
end
endmodule
```

Testbench:

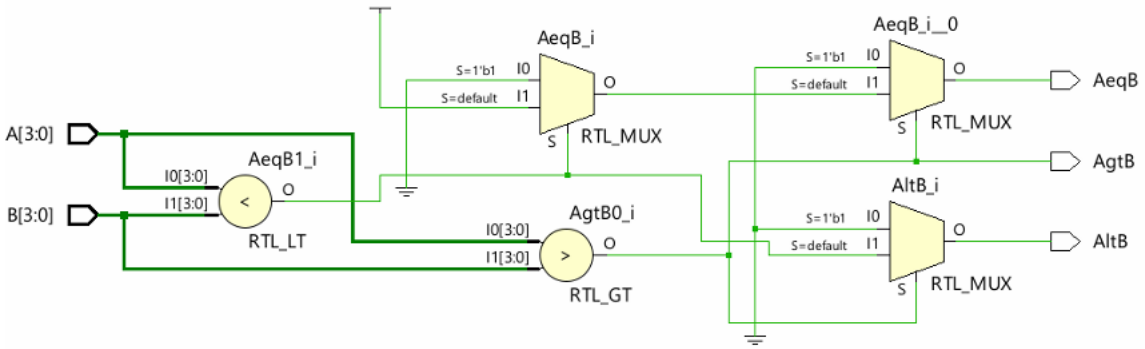
```
module ALU_tb;
reg [3:0] A, B;
reg [2:0] ch;
wire [3:0] result;
wire carry;
ALU uut (
.A(A),.B(B),.ch(ch),.result(result),.carry(carry));
initial begin
$monitor("Time=%0t | A=%b B=%b ch=%b | result=%b carry=%b",
$time, A, B, ch, result, carry);
// Operation 1: Addition
A = 4'b1100; B = 4'b0111; ch = 3'b000;
#10;
// Operation 2: Subtraction (with borrow)
A = 4'b0011; B = 4'b1001; ch = 3'b001;
#10;
// Operation 3: Bitwise AND
A = 4'b1100; B = 4'b1010; ch = 3'b010;
#10;
// Operation 4: Bitwise OR
A = 4'b1100; B = 4'b1010; ch = 3'b011;
#10;
// Operation 5: Bitwise XOR
A = 4'b1100; B = 4'b1010; ch = 3'b100;
#10;
// Operation 6: Bitwise NOT of A
A = 4'b1100; B = 4'b0000; ch = 3'b101;
#10;
// Operation 7: Left shift A by 1
A = 4'b0101; B = 4'b0000; ch = 3'b110;
#10;
// Operation 8: Right shift A by 1
A = 4'b0101; B = 4'b0000; ch = 3'b111;
#10;
$finish;
end
endmodule
```

Waveform:



2. Comparator

Design:



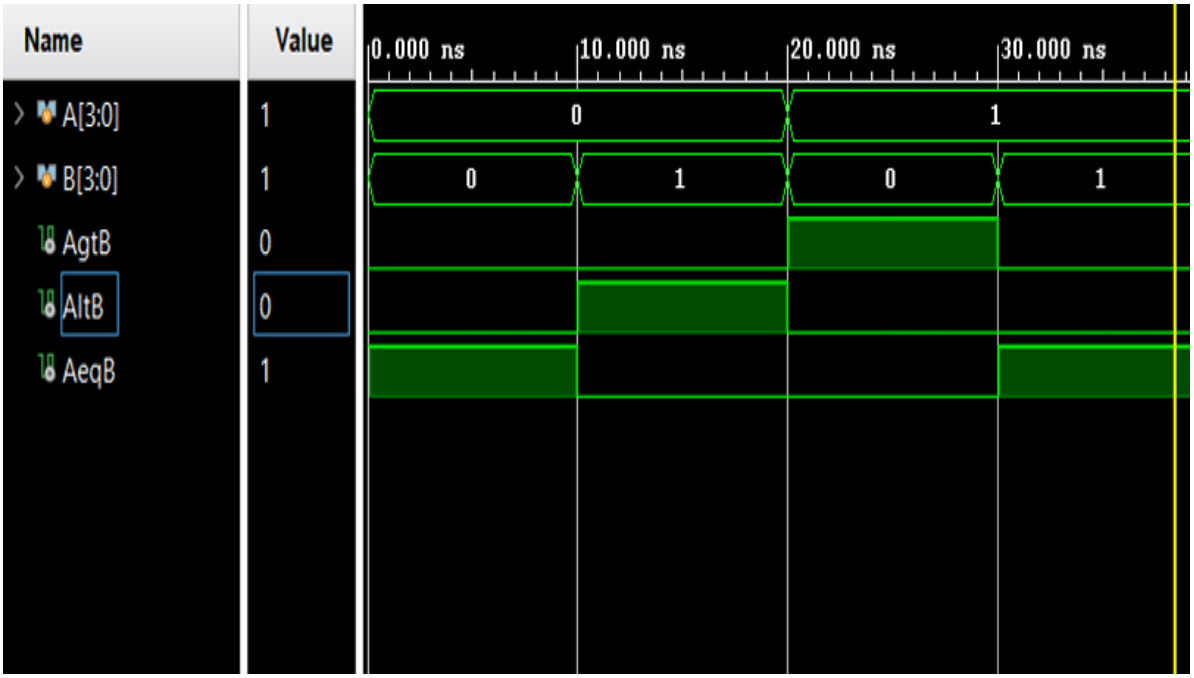
Code:

```
module Multibit_Comp(A,B,AgtB,AeqB,AltB);
parameter N = 4;
input [N-1:0]A,B;
output reg AgtB,AeqB,AltB;
always@(A,B)
if (A>B)
begin
AgtB=1;
AeqB=0;
AltB=0;
end
else if(A<B)
begin
AgtB=0;
AeqB=0;
AltB=1;
end
else
begin
AgtB=0;
AeqB=1;
AltB=0;
end
endmodule
```

Testbench:

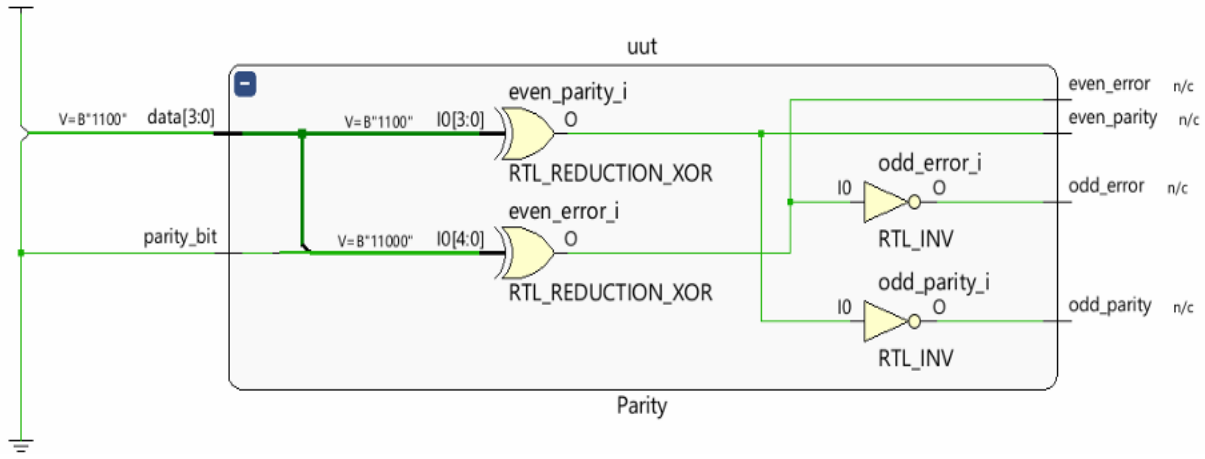
```
1  module Multibit_Comp_tb;
2  reg [3:0]A,B;
3  wire AgtB,AltB,AeqB;
4  Multibit_Comp A1(A,B,AgtB,AeqB,AltB);
5  initial begin
6  A=4'b0000;B=4'b0000;
7  #10 A=4'b0000;B=4'b0001;
8  #10 A=4'b0001;B=4'b0000;
9  #10 A=4'b0001;B=4'b0001;
10 #10 $finish;
11 end
12 endmodule
```

Waveform:



3. 4-Bit Parity Generator and Checker

Design:



Code:

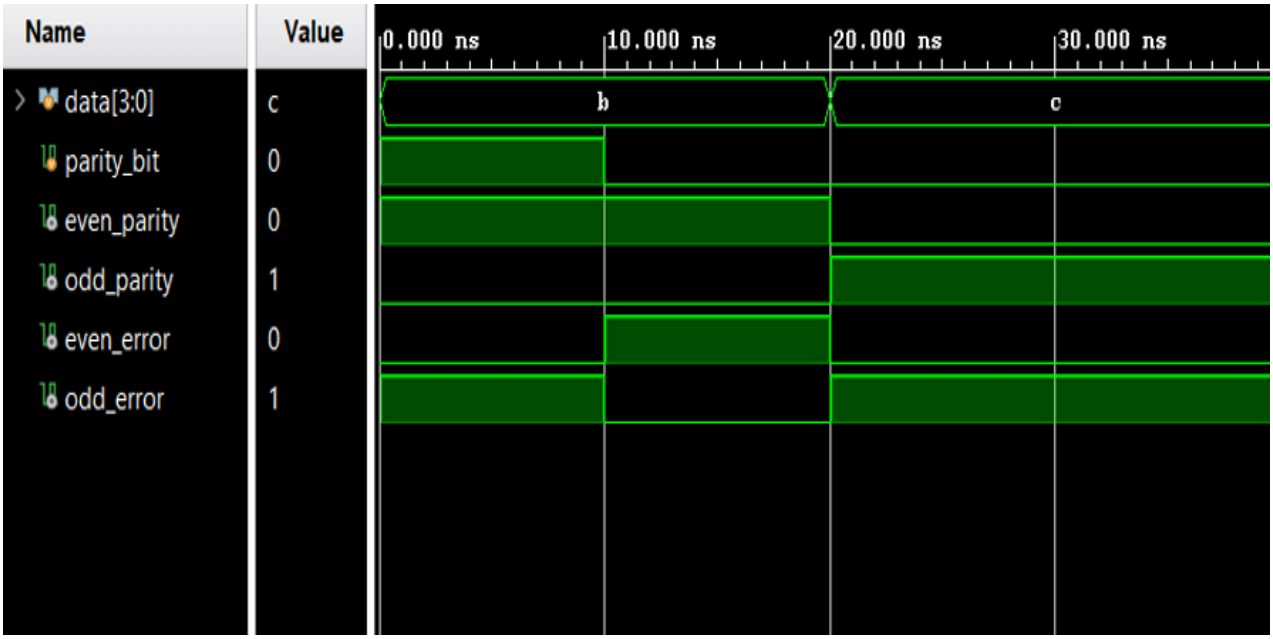
```
module Parity (input [3:0] data,input parity_bit,output even_parity,
odd_parity,
even_error, odd_error);
// Generate even parity bit
assign even_parity = ^data;
// Generate odd parity bit
assign odd_parity = ~(^data);
// Check even parity: 0 means correct, 1 means error
assign even_error = ^{data, parity_bit};
// Check odd parity: 0 means correct, 1 means error
assign odd_error = ~(^({data, parity_bit}));
```

Testbench:

```
module Parity_tb;
reg [3:0] data;
reg parity_bit;
wire even_parity,odd_parity,even_error,odd_error;
Parity uut (.data(data),.parity_bit(parity_bit), .even_parity(even_parity),
.odd_parity(odd_parity),.even_error(even_error),.odd_error(odd_error));
initial begin
$monitor("Time=%0t | data=%b | parity_bit=%b | even_parity=%b | odd_parity=%b |

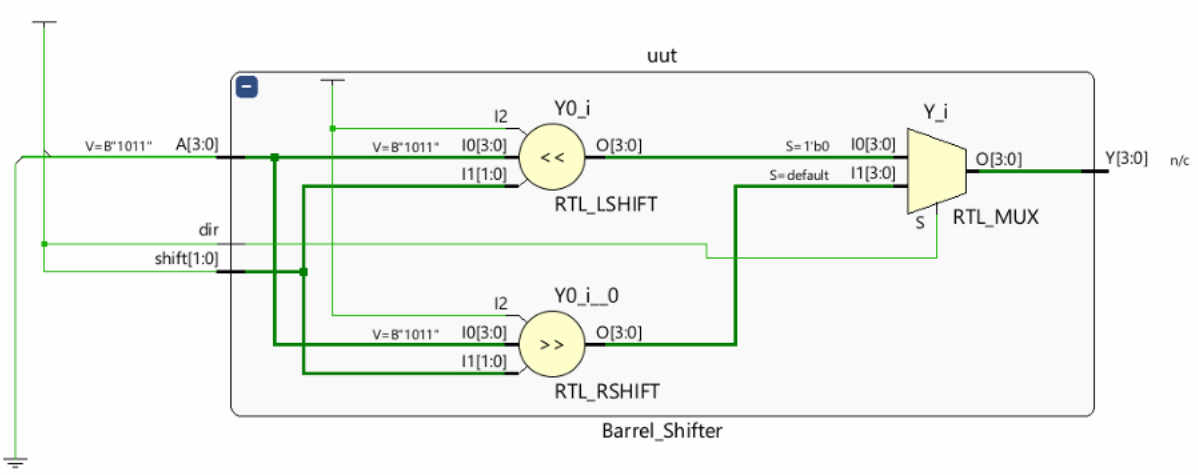
even_error=%b | odd_error=%b", $time, data, parity_bit,even_parity, odd_parity,
even_error,
odd_error);
// Test 1: Three ones; even parity bit should be 1
data = 4'b1011;
parity_bit = 1'b1;
#10;
// Test 2: Same data with incorrect even parity bit
data = 4'b1011;
parity_bit = 1'b0;
#10;
// Test 3: Two ones; even parity bit should be 0
data = 4'b1100;
parity_bit = 1'b0;
#10;
// Test 4: Same data with incorrect odd parity bit
data = 4'b1100;
parity_bit = 1'b0;
#10;
$finish;
end
endmodule
```

Waveform:



4. 4-Bit Barrel Shifter

Design:



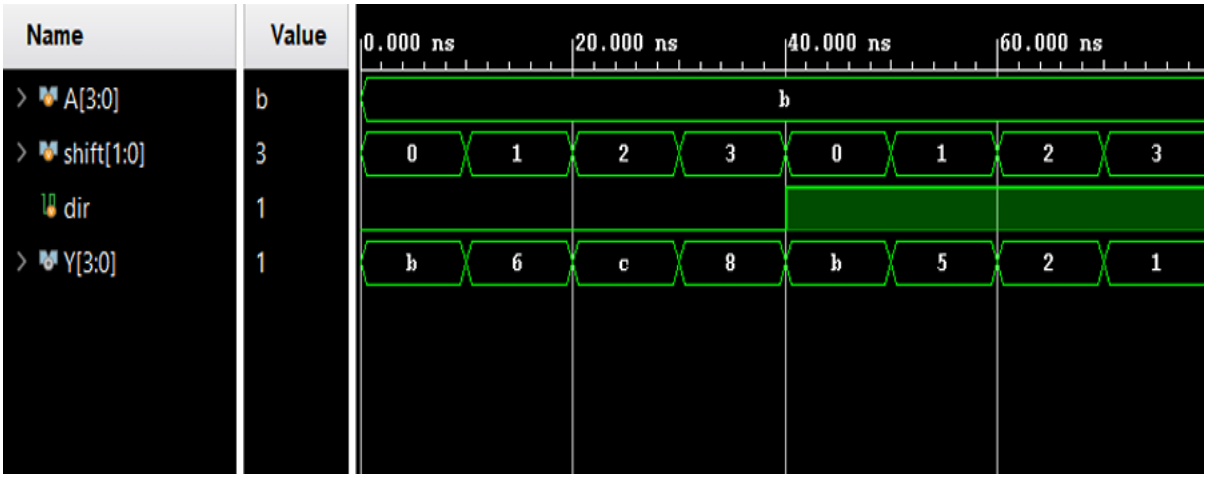
Code:

```
module Barrel_Shifter(
input [3:0] A,
input [1:0] shift,
input dir,
// 0 = left shift, 1 = right shift
output reg [3:0] Y);
always @(*) begin
if (dir == 1'b0)
Y = A << shift;
else
Y = A >> shift;
end
endmodule
```

Testbench:

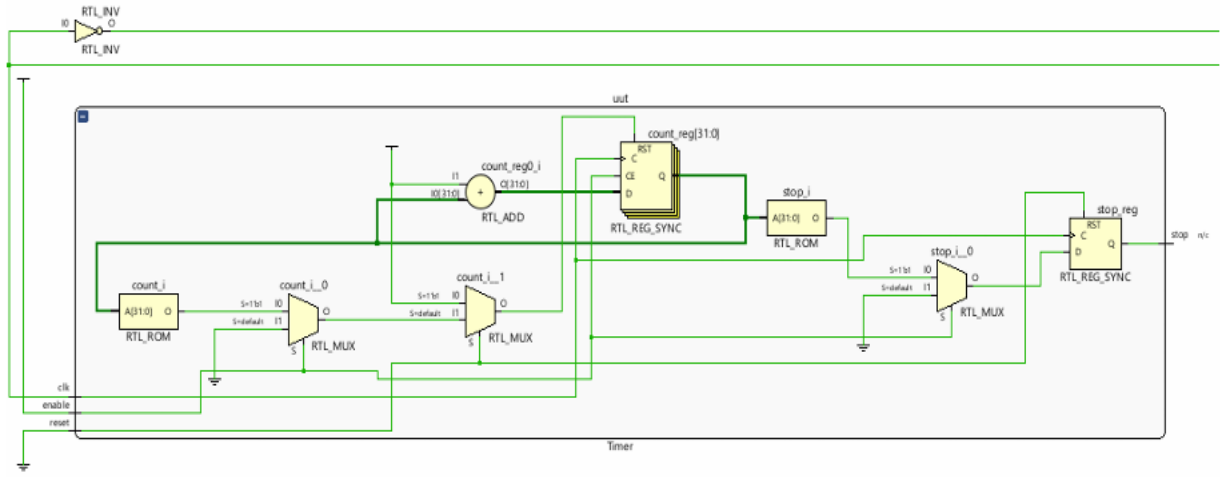
```
module Barrel_Shifter_tb;
reg [3:0] A;
reg [1:0] shift;
reg dir;
wire [3:0] Y;
Barrel_Shifter uut (.A(A),.shift(shift),.dir(dir),.Y(Y));
initial begin
$monitor("Time=%0t | A=%b | dir=%b | shift=%d | Y=%b",
$time, A, dir, shift, Y);
// Left shift by 0
A = 4'b1011; dir = 1'b0; shift = 2'd0;
#10;
// Left shift by 1
shift = 2'd1;
#10;
// Left shift by 2
shift = 2'd2;
#10;
// Left shift by 3
shift = 2'd3;
#10;
// Right shift by 0
A = 4'b1011; dir = 1'b1; shift = 2'd0;
#10;
// Right shift by 1
shift = 2'd1;
#10;
// Right shift by 2
shift = 2'd2;
#10;
// Right shift by 3
shift = 2'd3;
#10;
$finish;
end
endmodule
```

Waveform:



5. Timer Module

Design:



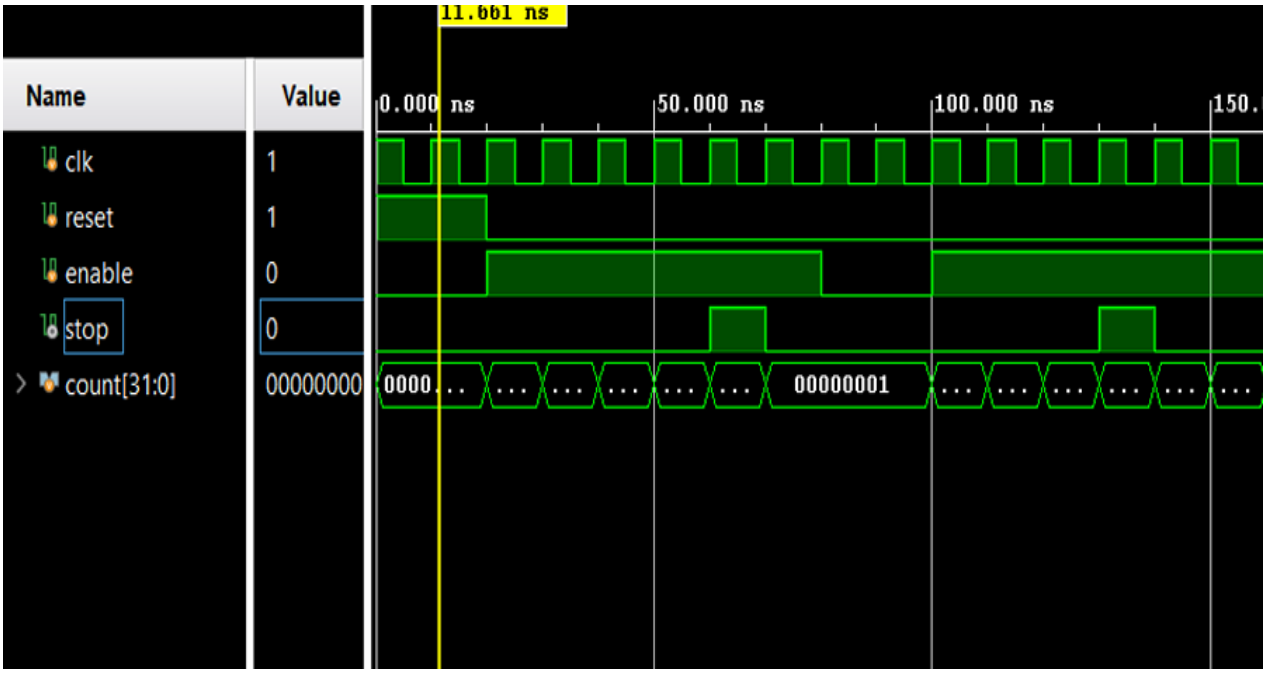
Code:

```
module Timer #(parameter COUNT_MAX = 5)(
        input clk,reset, enable,
        output reg stop
    );
        reg [31:0] count;
        always @(posedge clk) begin
            if (reset) begin
                count <= 0;
                stop <= 0;
            end
            else if (enable) begin
                if (count == COUNT_MAX - 1) begin
                    count <= 0;
                    stop <= 1;
                end
                else begin
                    count <= count + 1;
                    stop <= 0;
                end
            end
            else begin
                stop <= 0;
            end
        end
    endmodule
```

Testbench:

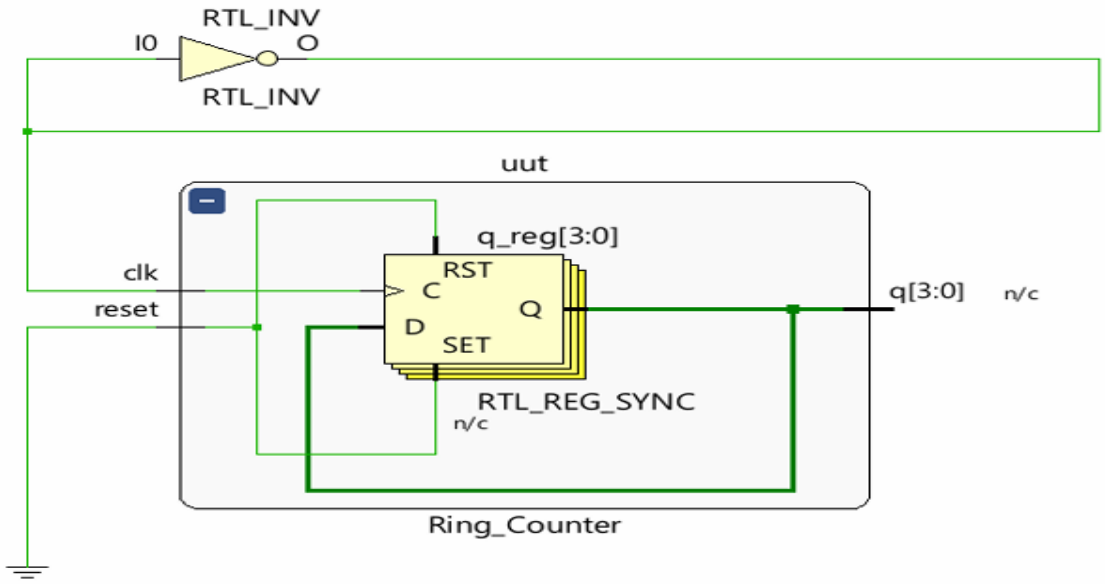
```
module Timer_tb;
reg clk,reset,enable;
wire stop;
Timer #(.COUNT_MAX(5)) uut (
.clk(clk),.reset(reset), .enable(enable),.stop(stop));
always #5 clk = ~clk;
initial begin
$monitor("Time=%0t | reset=%b enable=%b done=%b",
$time, reset, enable, done);
clk = 0;
reset = 1;
enable = 0;
#10;
reset = 0;
enable = 1;
#70;
enable = 0;
#20;
enable = 1;
#40;
$finish;
end
endmodule
```

Waveform:



6. Ring Counter

Design:



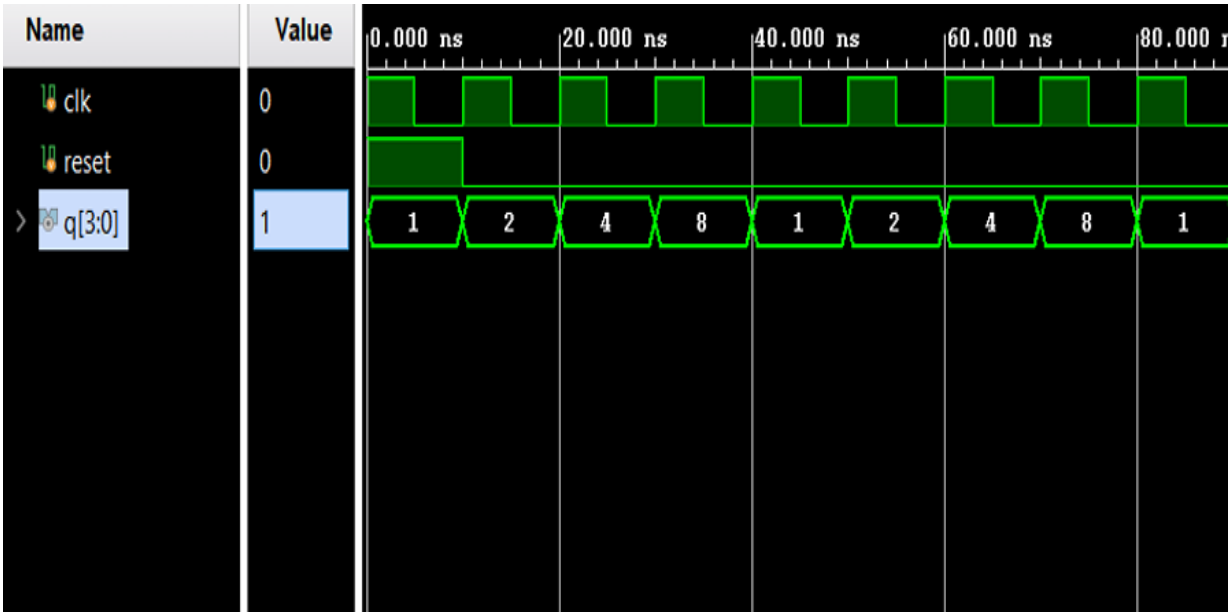
Code:

```
module Ring_Counter (
input clk,reset,
output reg [3:0] q);
always @(posedge clk) begin
if (reset)
q <= 4'b0001;
else
q <= {q[2:0], q[3]};
end
endmodule
```

Testbench:

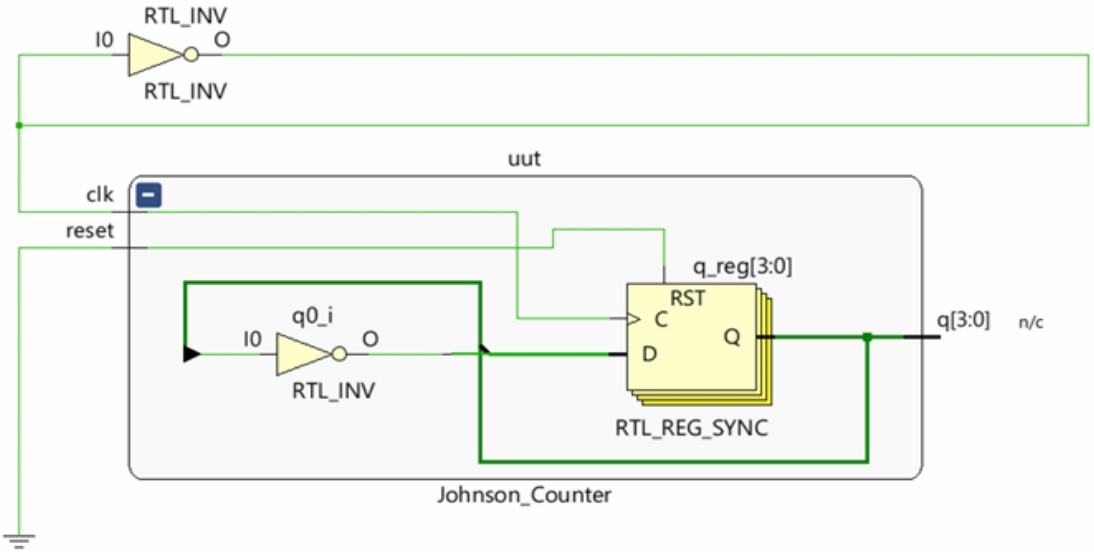
```
module Ring_Counter_tb;
reg clk, reset;
wire [3:0] q;
Ring_Counter uut (.clk(clk),.reset(reset),.q(q));
always #5 clk = ~clk;
initial begin
$monitor("Time=%0t | reset=%b | q=%b",
$time, reset, q);
clk = 0;
reset = 1;
#10;
reset = 0;
#80;
$finish;
end
endmodule
```

Waveform:



7. Johnson Counter

Design:



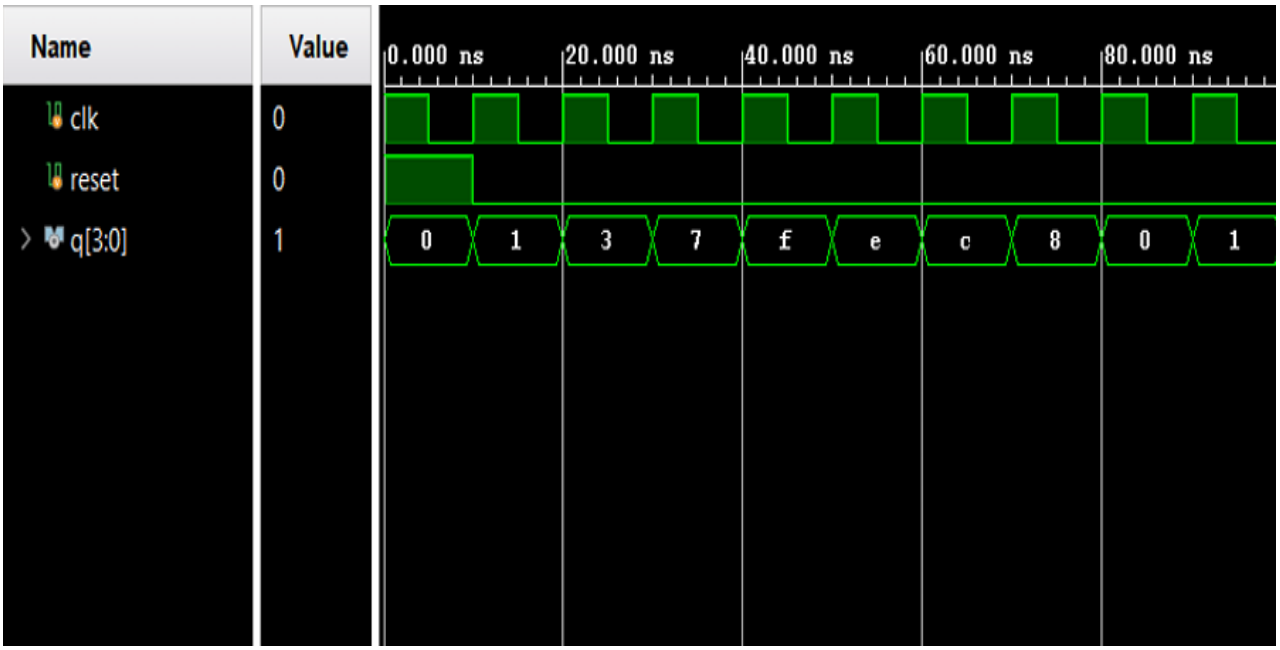
Code:

```
module Johnson_Counter (
input clk,reset,
output reg [3:0] q);
always @(posedge clk) begin
if (reset)
q <= 4'b0000;
else
q <= {q[2:0], ~q[3]};
end
endmodule
```

Testbench:

```
module Johnson_Counter_tb;
reg clk, reset;
wire [3:0] q;
Johnson_Counter uut (.clk(clk),.reset(reset),.q(q));
always #5 clk = ~clk;
initial begin
$monitor("Time=%0t | reset=%b | q=%b",
$time, reset, q);
clk = 1;
reset = 1;
#10;
reset = 0;
#90;
$finish;
end
endmodule
```

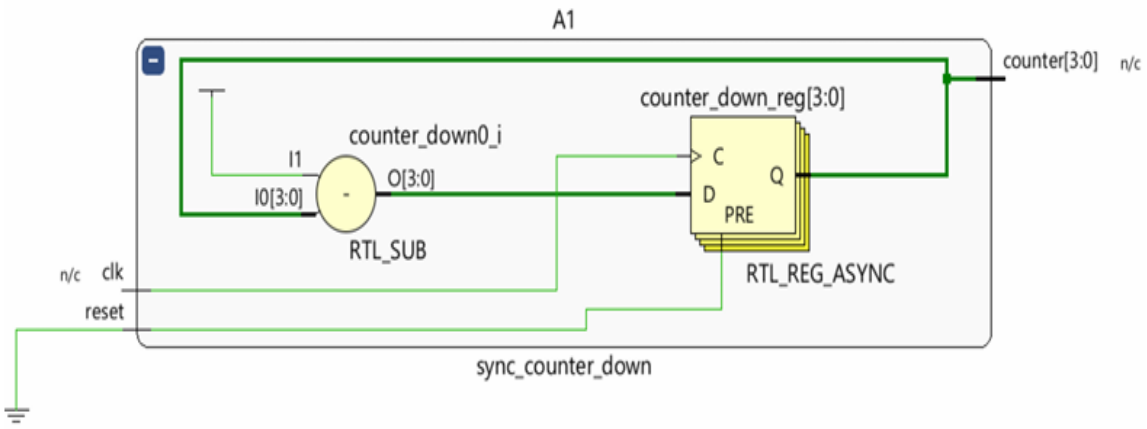
Waveform:



8. Synchronous and Asynchronous Counters

Synchronous counter

Design:



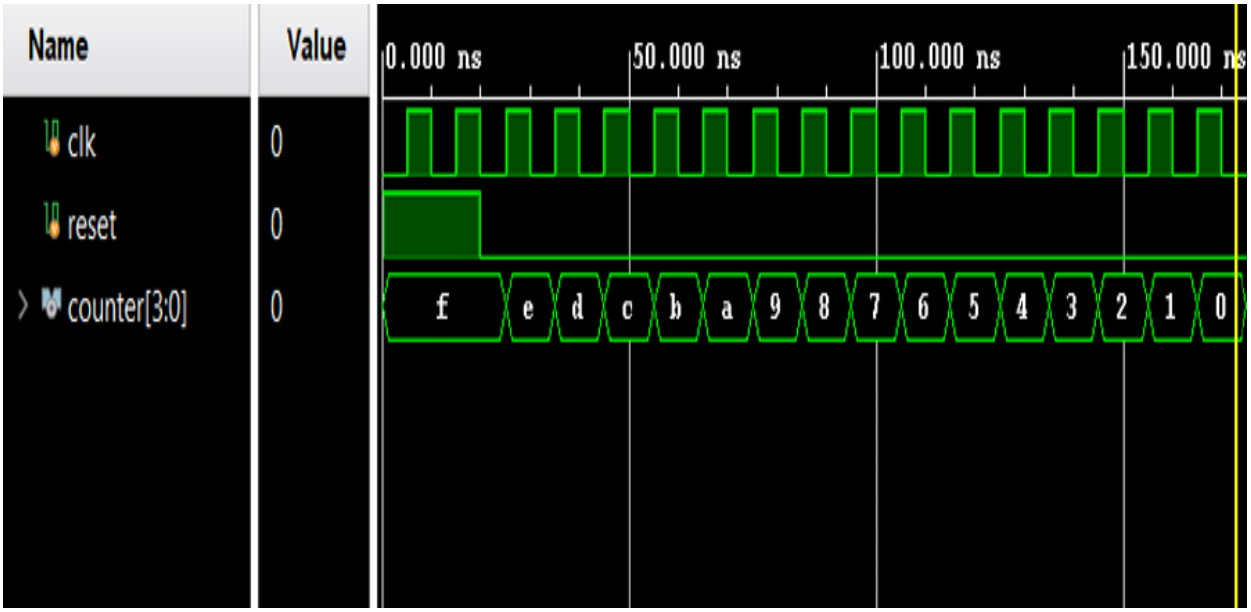
Code:

```
module sync_counter_down(
input clk,reset,
output [3:0] counter
);
reg [3:0]counter_down;
always@(posedge clk or posedge reset)
begin
if(reset)
counter_down <=4'd15;
else counter_down <= counter_down - 4'd1;
end
assign counter = counter_down;
endmodule
```

Testbench:

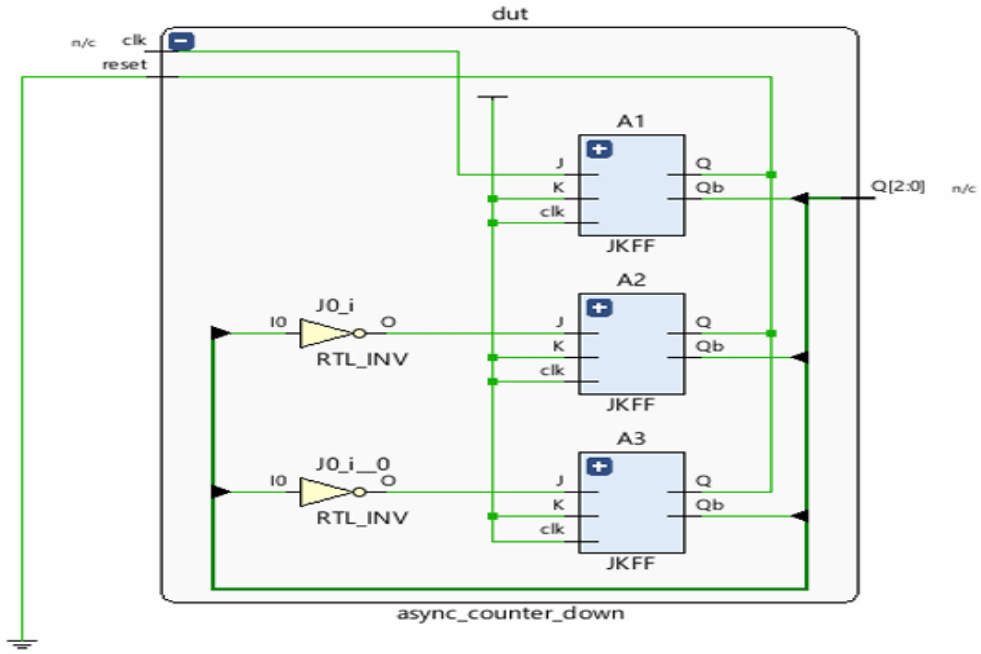
```
module sync_counter_down_tb;
reg clk, reset;
wire [3:0]counter;
sync_counter_down uut(clk,reset,counter);
initial begin
clk = 0;
forever #5 clk = ~clk;
end
initial begin
reset = 1;
#20 reset = 0;
#155 $finish;
end
endmodule
```

Waveform:



Asynchronous counter

Design:



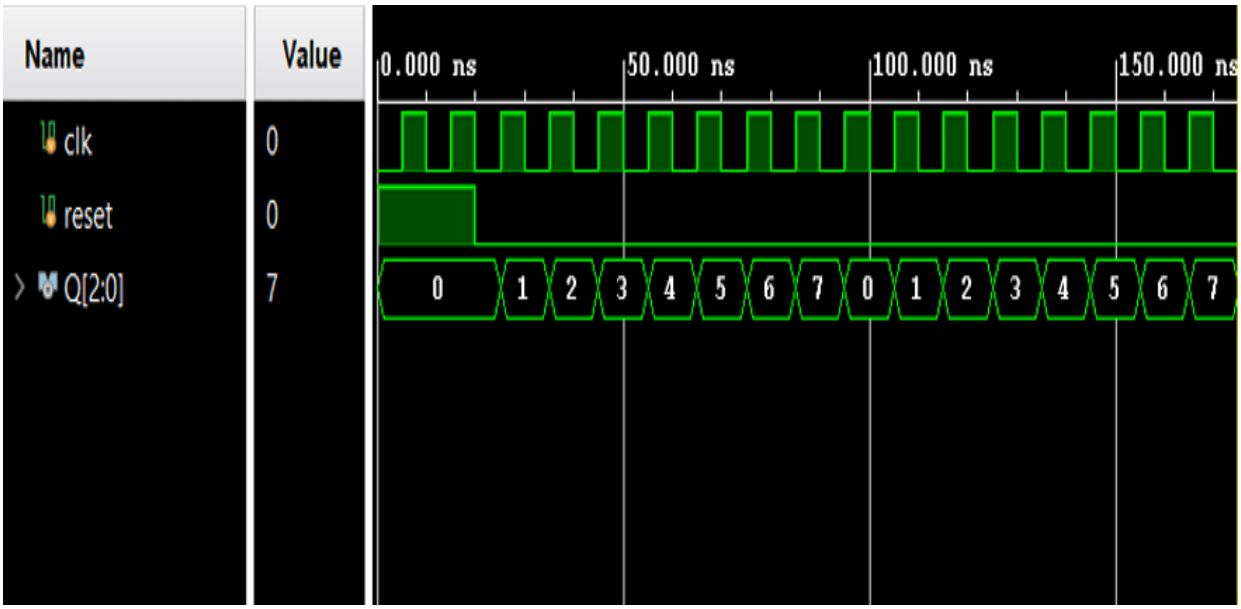
Code:

```
module async_counter_down (
input clk,
input reset,
output [2:0] Q
);
JKFF A1 (
.clk(clk), .J(1'b1), .K(1'b1),
.reset(reset), .Q(Q[0])
);
JKFF A2 (
.clk(~Q[0]), .J(1'b1), .K(1'b1),
.reset(reset), .Q(Q[1])
);
JKFF A3 (
.clk(~Q[1]), .J(1'b1), .K(1'b1),
.reset(reset), .Q(Q[2])
);
endmodule
module JKFF (
input clk,J,K,reset,
output reg Q);
always @(posedge clk or posedge reset) begin
if (reset)
Q <= 1'b0;
else begin
case ({J, K})
2'b00: Q <= Q;        // Hold
2'b01: Q <= 1'b0;     // Reset
2'b10: Q <= 1'b1;     // Set
2'b11: Q <= ~Q;       // Toggle
endcase
end
end
endmodule
```

Testbench:

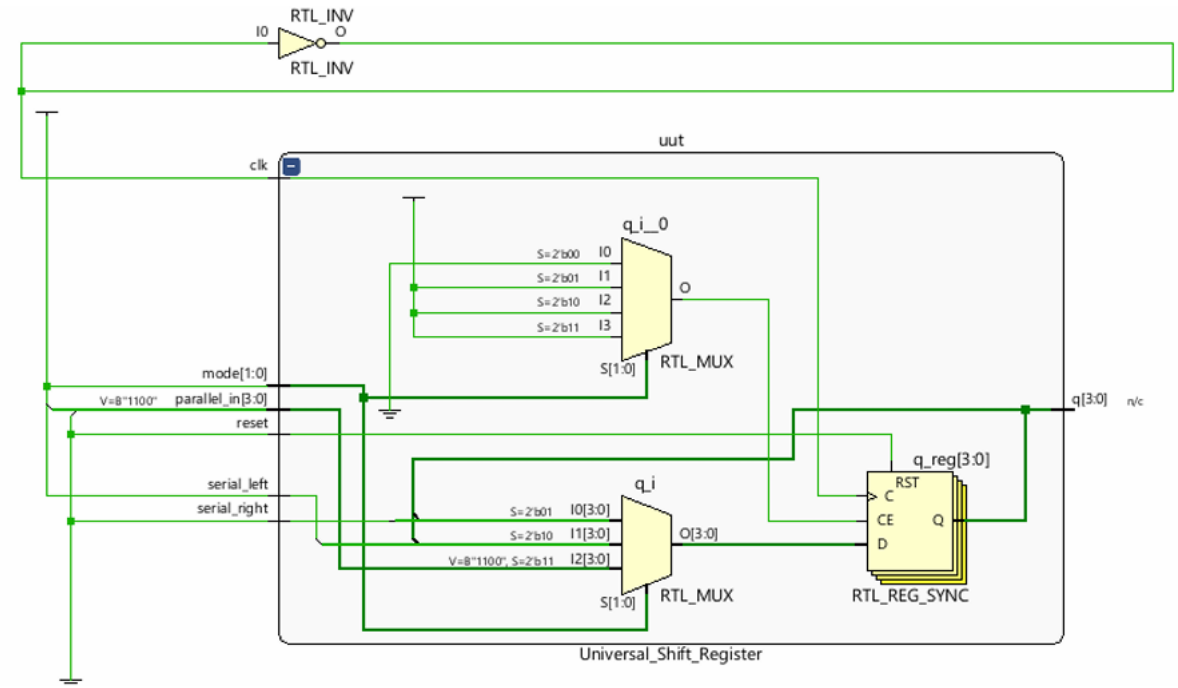
```
module async_counter_down_tb;
reg clk, reset;
wire [3:0]counter;
async_counter_down uut(clk,reset,counter);
initial begin
clk = 0;
forever #5 clk = ~clk;
end
initial begin
reset = 1;
#20 reset = 0;
#155 $finish;
end
endmodule
```

Waveform:



9. Shift Register

Design:



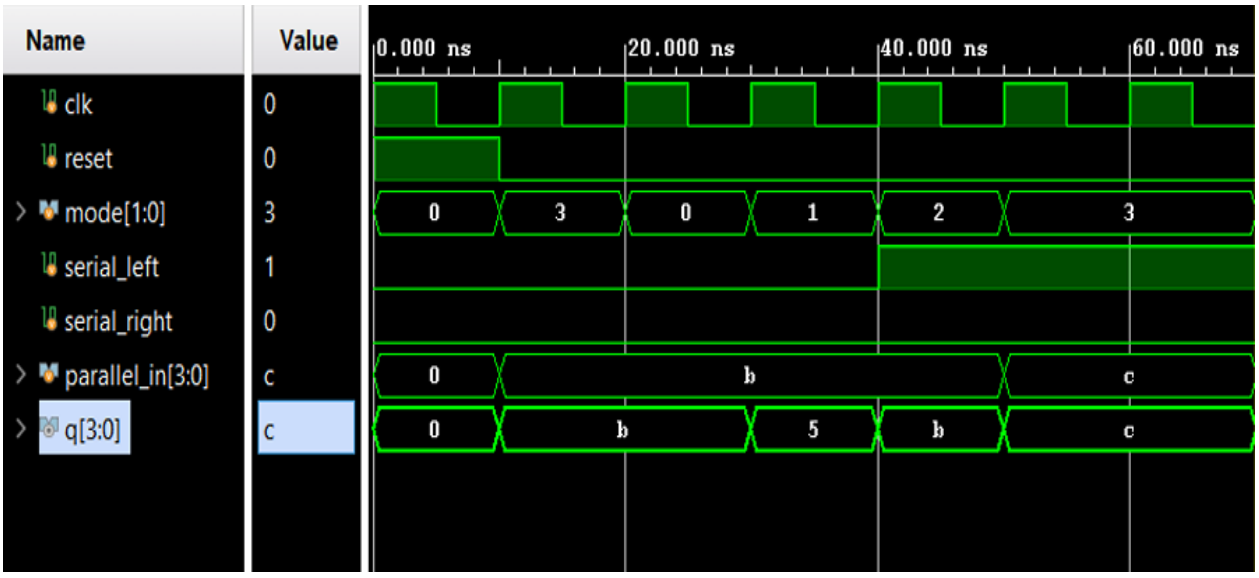
Code:

```
module Universal_Shift_Register (
input clk,reset,serial_left, serial_right,
input [1:0] mode,
input [3:0] parallel_in,
output reg [3:0] q
);
always @(posedge clk) begin
if (reset)
q <= 4'b0000;
else begin
case (mode)
2'b00: q <= q;
// Hold
2'b01: q <= {serial_right, q[3:1]};  // Shift right
2'b10: q <= {q[2:0], serial_left};   // Shift left
2'b11: q <= parallel_in;
endcase
end
end
endmodule
```

Testbench:

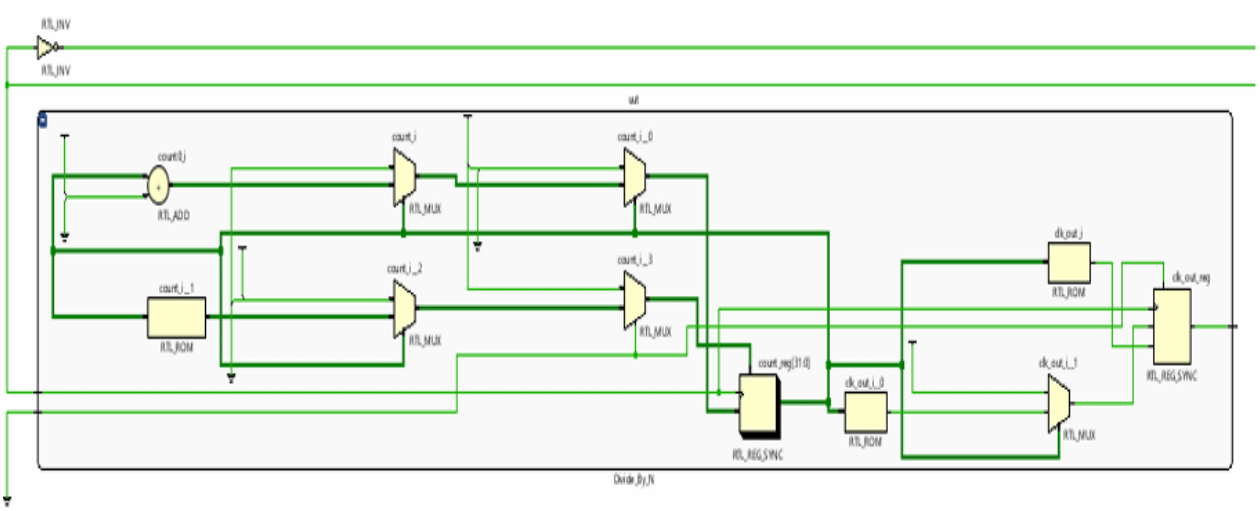
```
module Universal_Shift_Register_tb;
reg clk, reset;
reg [1:0] mode;
reg serial_left, serial_right;
reg [3:0] parallel_in;
wire [3:0] q;
// Parallel load
Universal_Shift_Register uut
(.clk(clk),.reset(reset),.mode(mode),.serial_left(serial_left),
.serial_right(serial_right),.parallel_in(parallel_in),.q(q));
always #5 clk = ~clk;
initial begin
$monitor("Time=%0t | mode=%b | parallel_in=%b | q=%b",
$time, mode, parallel_in, q);
clk = 1;
reset = 1;
mode = 2'b00;
serial_left = 0;
serial_right = 0;
parallel_in = 4'b0000;
#10 reset = 0;
// Parallel load 1011
mode = 2'b11;
parallel_in = 4'b1011;
#10;
// Hold
mode = 2'b00;
#10;
// Shift right
mode = 2'b01;
serial_right = 0;
#10;
// Shift left
mode = 2'b10;
serial_left = 1;
#10;
// Parallel load another value
mode = 2'b11;
parallel_in = 4'b1100;
#10;
#10 $finish;
end
endmodule
```

Waveform:



10. Divide-by-N Counter

Design:



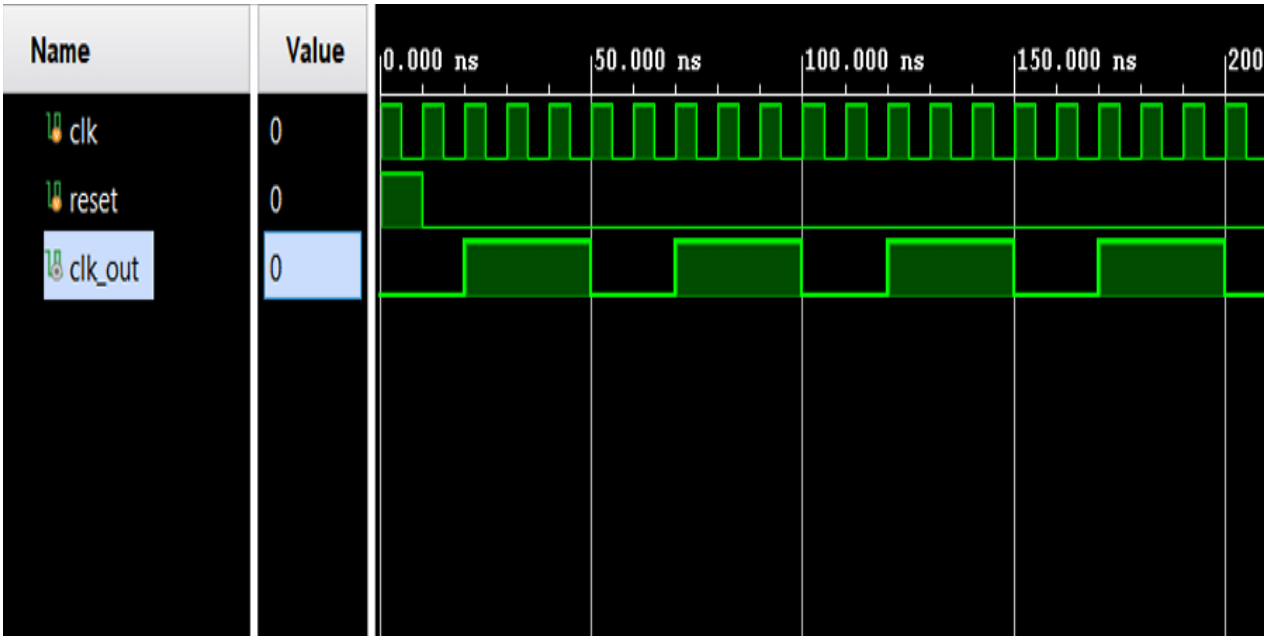
Code:

```
module Divide_By_N #(parameter N = 5)(
input clk,reset,
output reg clk_out);
integer count;
always @(posedge clk) begin
if (reset) begin
count <= 0;
clk_out <= 0;
end
else begin
if (count == (N/2)-1) begin
count <= count + 1;
clk_out <= 1;
end
else if (count == N-1) begin
count <= 0;
clk_out <= 0;
end
else begin
count <= count + 1;
end
end
end
endmodule
```

Testbench:

```
module Divide_By_N_tb;
reg clk, reset;
wire clk_out;
Divide_By_N #(.N(5)) uut (.clk(clk),.reset(reset),.clk_out(clk_out) );
always #5 clk = ~clk;
initial begin
$monitor("Time=%0t | reset=%b | count=%d | clk_out=%b",
$time, reset, uut.count, clk_out);
clk = 1;
reset = 1;
#10 reset = 0;
#200 $finish;
end
endmodule
```

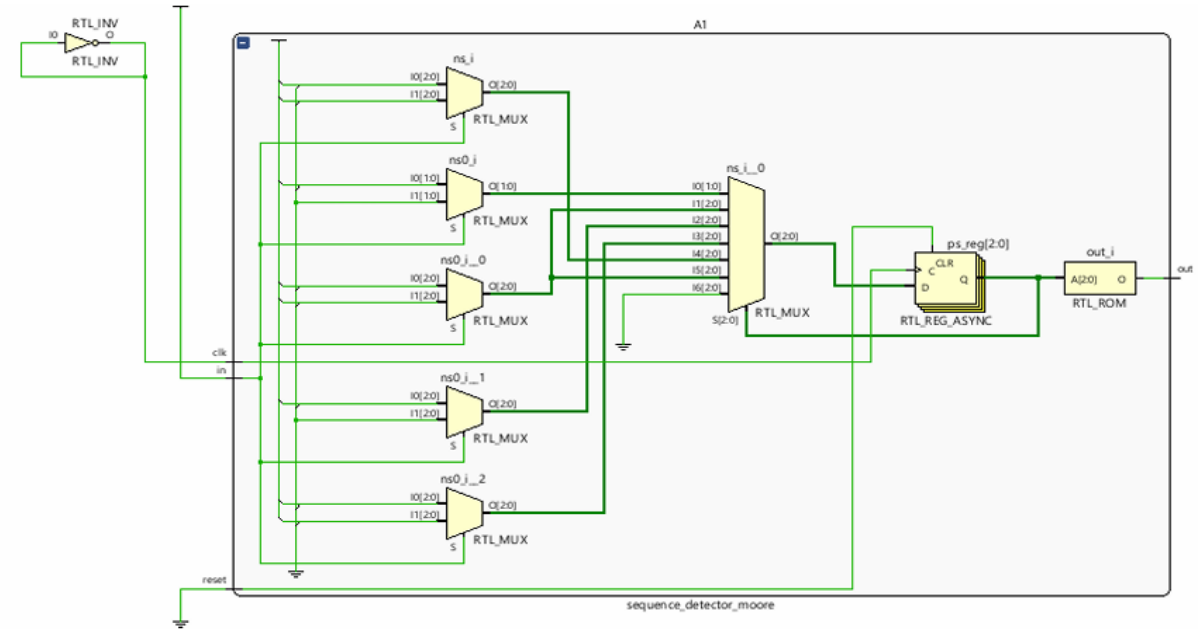
Waveform:



11. Mealy and Moore FSMs (one example each)

Moore (1011)

Design:



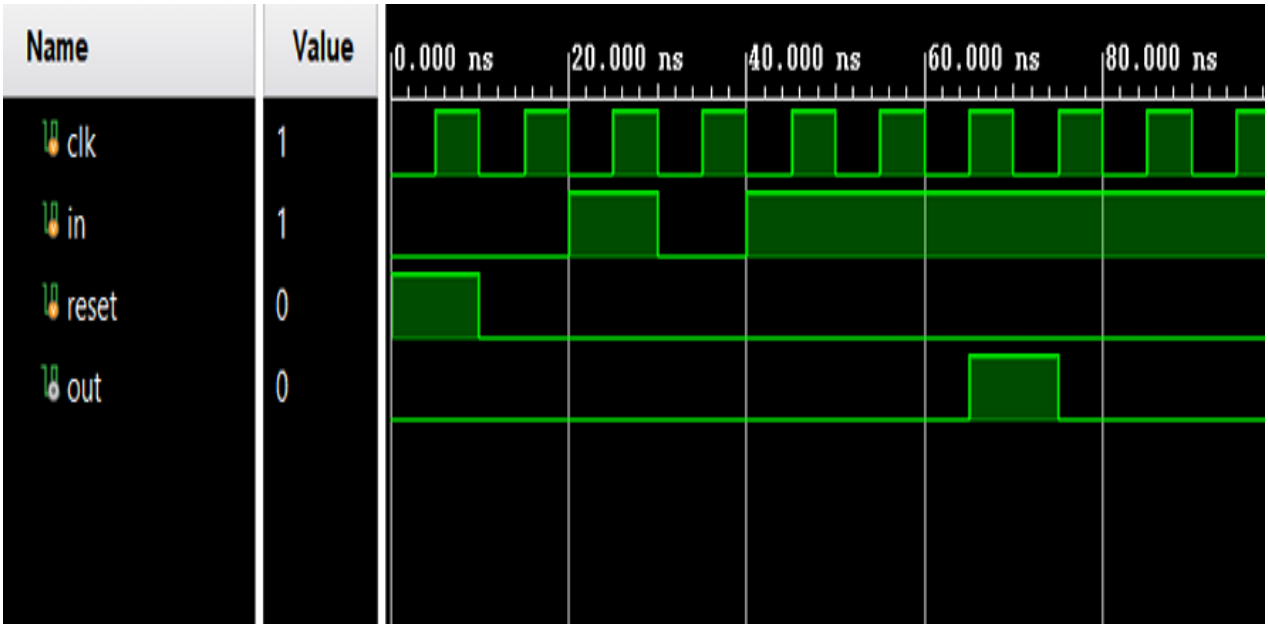
Code:

```
module sequence_detector_moore(input wire clk,reset,in,
output reg out
);
reg [2:0]ps,ns; // State declaration
//State encoding
parameter s0 = 3'b000,
s1 = 3'b001,
s2 = 3'b010,
s3 = 3'b011,
s4 = 3'b100,
s5 = 3'b101;
always@(posedge clk or posedge reset)
begin
if(reset)
ps<=s0;
else
ps <= ns;
end
always@(*)
begin
ns=ps;
case(ps)
s0:ns = in?s1:s0;
s1:ns = in?s1:s2;
s2:ns = in?s3:s0;
s3:ns = in?s4:s2;
s4:begin
if(in)
ns=s5;
else
ns=s2;
end
s5:begin
ns=in?s1:s2;
end
default:ns=s0;
endcase
end
always@(*)begin
case(ps)
s5:out = 1'b1;
default: out = 1'b0;
endcase
end
endmodule
```

Testbench:

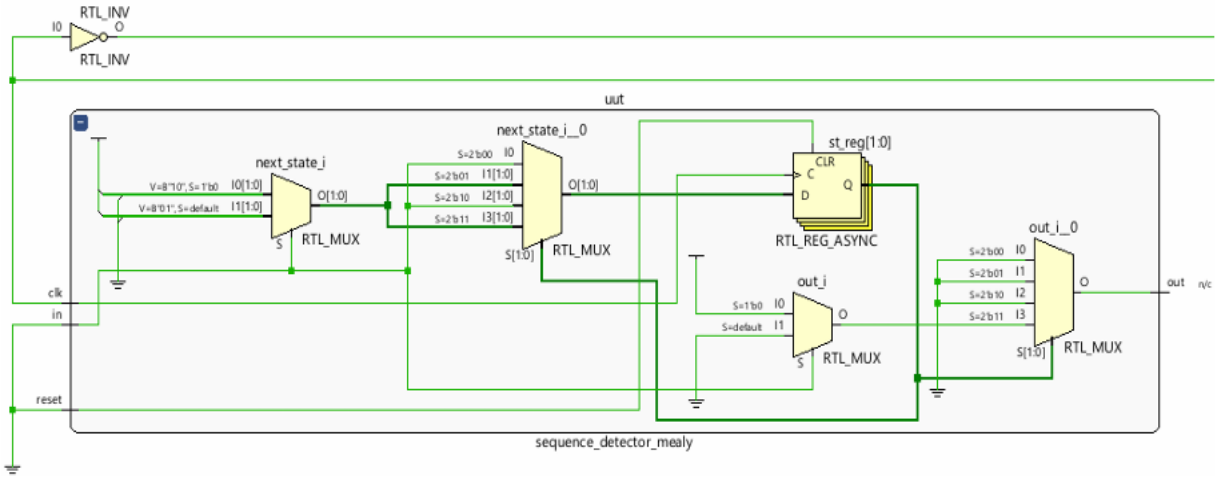
```
module sequence_detector_moore_tb;
reg clk,in,reset;
wire out;
sequence_detector_moore A1(clk,reset,in,out);
initial begin
clk = 0; reset = 1; in = 0;
#10 reset = 0; in = 0;
#10 in = 1;
#10 in = 0;
#10 in = 1;
#10 in = 1;
#50 $finish;
end
always #5 clk = ~clk;
endmodule
```

Waveform:



Mealy (1010)

Design:



Code:

```
module sequence_detector_mealy(
input wire clk, reset, in,
output reg out
);
reg [1:0] st, next_state;
parameter s0 = 2'b00,
s1 = 2'b01,
s2 = 2'b10,
s3 = 2'b11;
always @(posedge clk or posedge reset) begin
if (reset)
st <= s0;
else
st <= next_state;
end

     always @(*) begin
          next_state = st;
          out = 1'b0;

          case (st)
               s0: begin
                    if (in)
                         next_state = s1;
                    else
                         next_state = s0;
               end

               s1: begin
                    if (in == 0)
                         next_state = s2;
                    else
                         next_state = s1;
               end

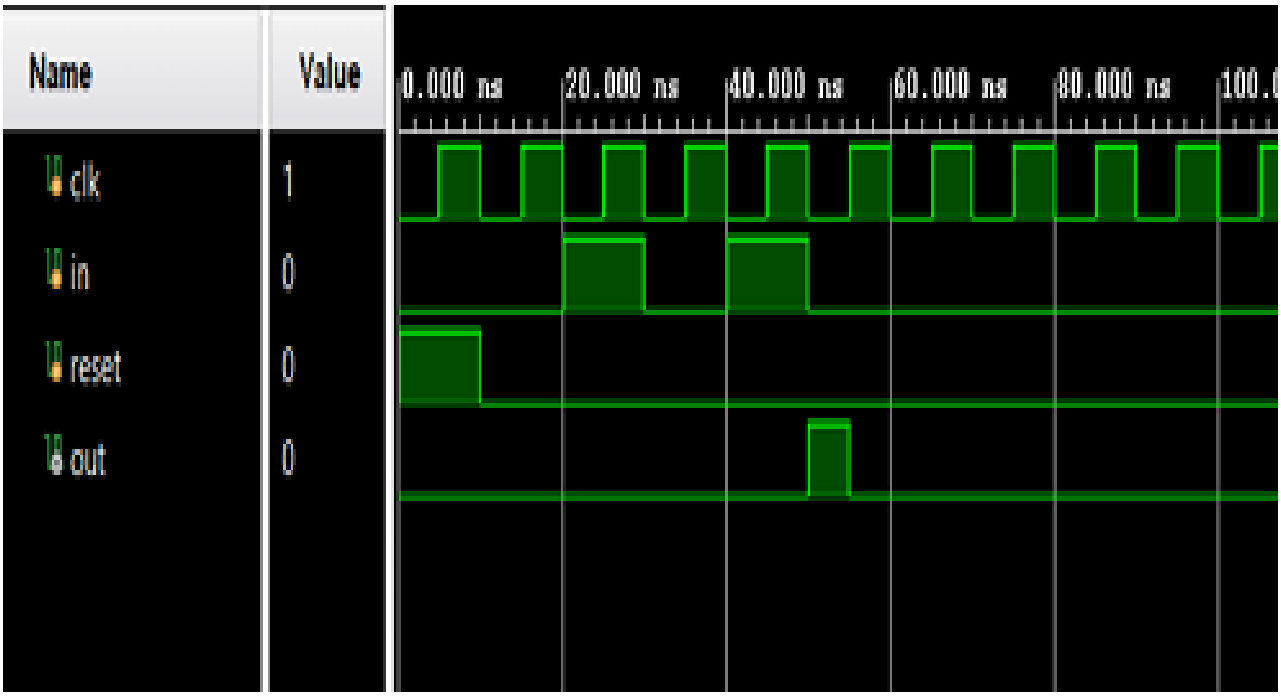
               s2: begin
                    if (in)
                         next_state = s3;
                    else
                         next_state = s0;
               end

               s3: begin
                    if (in == 0) begin
                         next_state = s2;
                         out = 1'b1;
                    end
                    else
                         next_state = s1;
               end
               default: next_state = s0;
          endcase
     end
endmodule
```

Testbench:

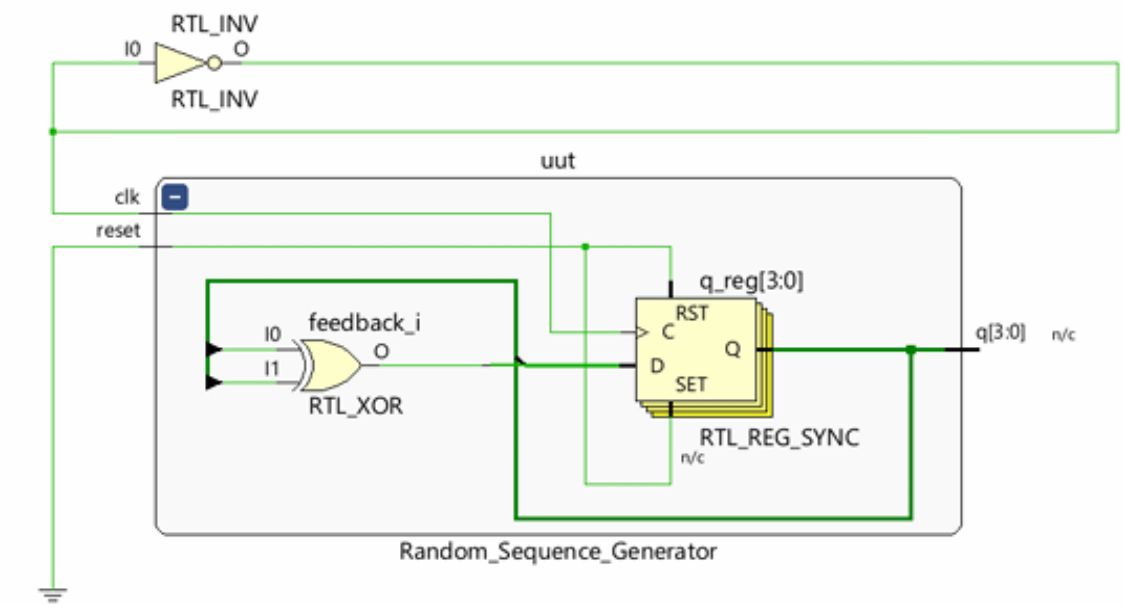
```
module sequence_detector_mealy_tb;
reg clk,in,reset;
wire out;
sequence_detector_mealy uut(clk,reset,in,out);
initial begin
clk = 0; reset = 1; in = 0;
#10 reset = 0; in = 0;
#10 in = 1;
#10 in = 0;
#10 in = 1;
#10 in = 0;
#10 in = 0;
#50 $finish;
end
always #5 clk = ~clk;
endmodule
```

Waveform:



12. Random Sequence Generator

Design:



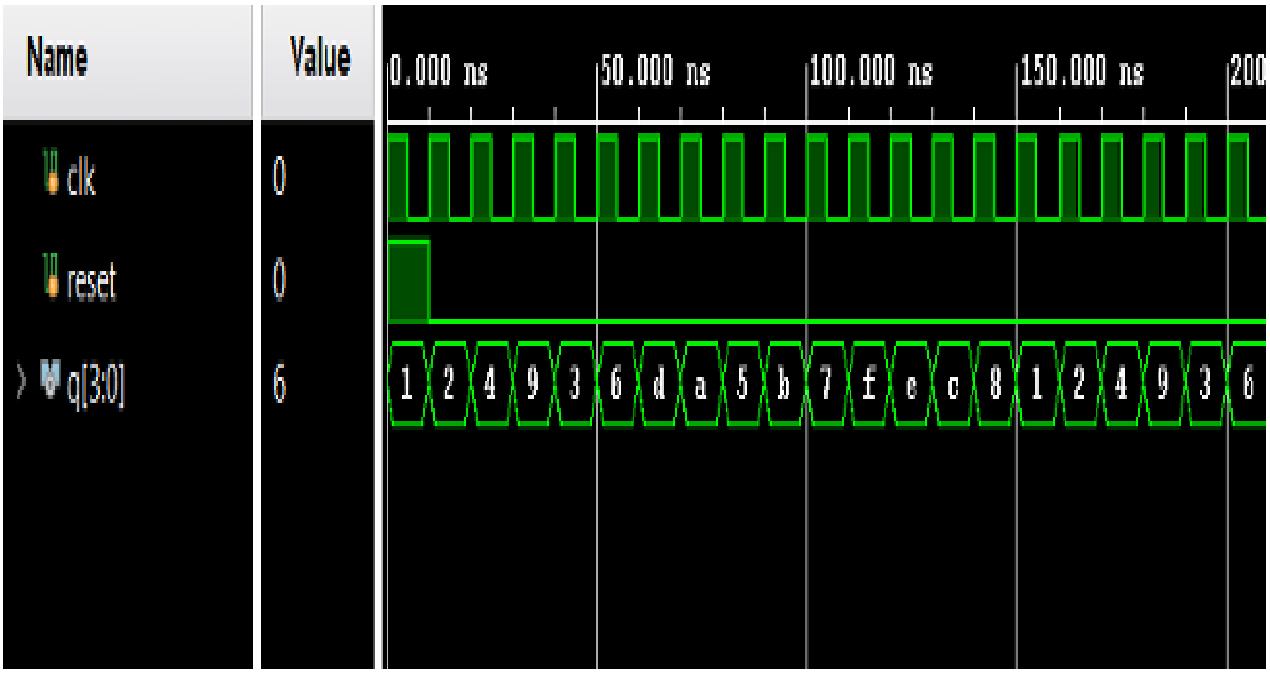
Code:

```
module Random_Sequence_Generator (
input clk,
input reset,
output reg [3:0] q
);
wire feedback;
// XOR feedback from selected register bits
assign feedback = q[3] ^ q[2];
always @(posedge clk) begin
if (reset)
q <= 4'b0001;  // Nonzero initial seed
else
q <= {q[2:0], feedback};
end
endmodule
```

Testbench:

```
module Random_Sequence_Generator_tb;
reg clk, reset;
wire [3:0] q;
Random_Sequence_Generator uut (.clk(clk),.reset(reset),.q(q) );
always #5 clk = ~clk;
initial begin
$monitor("Time=%0t | reset=%b | q=%b",
$time, reset, q);
clk = 1;
reset = 1;
#10 reset = 0;
#200 $finish;
end
endmodule
```

Waveform:



ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Prof. Ajaykumar Devarapalli**, for his valuable guidance and support throughout the completion of this Alternative Assessment Tool (AAT) report for the course **Advanced Digital and Logic Design (ADLD)**.

I am also thankful to the Department of Electronics and Communication Engineering, BMS College of Engineering, for providing the opportunity and academic environment to carry out this work on the design and verification of combinational and sequential digital modules using Verilog HDL.

Finally, I would like to acknowledge everyone who contributed directly or indirectly to the successful completion of this report.

Vikhyathi M
USN: 1BM24EC253